C05 - Assessment Tasks 2

Error Analysis Task

Name: M. Pranay

Reg No: 192525382

SET1

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

1. Hadoop Job Fails Because Input Splits Are Uneven and Reducers Remain Idle

Problem Analysis

In a Hadoop MapReduce job, the input data is divided into multiple input splits and processed by mapper tasks. If the input splits are not evenly distributed, some mapper tasks receive significantly more data than others. The larger tasks take longer to complete, while other mapper tasks finish early.

This creates a **data imbalance or data skew problem**. Since the reducers receive intermediate output from the mappers, reducers may remain idle while waiting for the slowest mapper tasks to finish. In severe cases, the job may fail because of excessive processing time, memory consumption, or task timeout.

Another possible design error is choosing an unsuitable HDFS block size or creating a small number of very large files. Poor partitioning of intermediate keys can also cause one reducer to receive much more data than other reducers.

Likely Design Errors

1. Unequal input file sizes.
2. Poor HDFS block or input-split configuration.
3. Data skew where some partitions contain much more data.
4. Incorrect number of reducers.
5. Poor key partitioning.
6. Extremely large individual records.
7. Lack of a suitable combiner for reducing intermediate data.

Corrective Actions

- Choose an appropriate HDFS block size according to the workload.
- Divide very large files into reasonably balanced files.
- Ensure input data is distributed across multiple HDFS blocks.
- Increase or decrease the number of reducers based on the amount of data.
- Use a custom partitioner when the default hash partitioner creates skew.
- Use a **Combiner** to reduce the amount of intermediate data transferred to reducers.
- Identify highly frequent keys and handle them separately if necessary.
- Monitor mapper and reducer execution times.
- Use Hadoop counters and logs to identify skewed partitions.

Expected Result

After correcting the partitioning and split configuration, mapper tasks should process approximately balanced amounts of data. Reducers will receive data more evenly and remain active instead of waiting for slow tasks. This improves cluster utilization, reduces execution time, and makes the Hadoop job more reliable.

2. HDFS Data Becomes Unavailable After a Single Node Failure

Problem Analysis

HDFS is designed to provide fault tolerance by storing multiple replicas of each data block on different DataNodes. If a single DataNode fails, another DataNode containing a replica should serve the required data.

If data becomes unavailable immediately after one node failure, the deployment probably has an incorrect **replication configuration**. For example, if the replication factor is set to 1, each block has only one copy. Failure of the DataNode containing that copy results in data unavailability.

Another possible problem is a **single point of failure at the NameNode**. The NameNode maintains filesystem metadata, including the locations of HDFS blocks. If only one NameNode exists and it fails, clients may not be able to access the HDFS namespace even when DataNodes are still operational.

Likely Design Errors

1. Replication factor is set too low.
2. Some blocks are under-replicated.
3. Replicas are stored on the same failure domain.
4. No rack-aware replica placement.
5. Single NameNode without High Availability.
6. Standby NameNode is not properly synchronized.
7. Automatic failover is not configured.
8. Failure monitoring is inadequate.

Corrective Actions

- Set the HDFS replication factor to an appropriate value, commonly **3** for production systems.
- Ensure replicas are distributed across multiple DataNodes.
- Use rack awareness so replicas are not concentrated in the same rack.

- Monitor under-replicated blocks.
- Configure **NameNode High Availability (HA)** using Active and Standby NameNodes.
- Configure automatic failover where appropriate.
- Monitor DataNode and NameNode health continuously.
- Use HDFS commands and monitoring tools to identify missing or under-replicated blocks.
- Replace failed hardware and allow HDFS to automatically re-replicate blocks.

Expected Result

With proper replication, failure of one DataNode should not make the data unavailable because another replica can serve the request. NameNode HA also removes the NameNode as a single point of failure. Therefore, the HDFS cluster becomes more fault tolerant, reliable, and highly available.

3. ETL and Apache NiFi Create Duplicate Records

Problem Analysis

Apache NiFi is commonly used for data ingestion and movement between different systems. Duplicate records can occur when the same data is processed multiple times. This may happen because a flow is restarted, a processor retries a failed operation, or the same input file is detected again.

For example, an ETL process may read a transaction file and successfully insert records into the analytics database. If NiFi does not correctly record that the file has already been processed, it may read the same file again and insert the records a second time.

Duplicate data can also occur when there is no unique identifier or constraint in the destination system.

Likely Causes

1. Same input file is processed multiple times.
2. NiFi processor retries an operation after a timeout.
3. Failure handling is incorrectly configured.
4. No unique transaction ID exists.
5. Destination database has no primary-key constraint.
6. ETL process performs repeated inserts instead of upserts.
7. NiFi state/checkpoint information is not properly maintained.

8. Multiple ingestion flows process the same source data.
9. Message delivery is repeated after a temporary failure.
10. Deduplication is not implemented.

Recommended Fixes

- Assign a unique ID to every transaction or record.
- Use appropriate NiFi state management for tracking processed data.
- Configure success, failure, and retry relationships correctly.
- Use suitable NiFi processors for duplicate detection and record filtering.
- Add primary-key or unique constraints to the destination database.
- Implement **upsert** operations instead of blindly inserting records.
- Store source file names, timestamps, and unique transaction IDs for tracking.
- Maintain checkpoints for incremental ingestion.
- Ensure that only one ingestion process handles a particular source.
- Monitor failed and retried FlowFiles.
- Periodically run data-quality checks to identify existing duplicates.

Example

Suppose a payment with transaction ID TX1001 is received twice. Without deduplication, the analytics database may contain:

TX1001, ₹500

TX1001, ₹500

With a unique transaction ID and an appropriate database constraint, the second occurrence can be rejected or used to update the existing record.

Expected Result

The ingestion pipeline becomes **idempotent**, meaning processing the same input more than once does not create additional incorrect records. This improves data quality and ensures that analytics and reports are based on accurate information.

4. YARN Cluster Suffers from Resource Starvation

Problem Analysis

YARN manages resources such as CPU and memory in a Hadoop cluster. When multiple users submit jobs at the same time, poor resource scheduling can cause one user or application to consume a large portion of the available resources.

For example, if a large MapReduce or Spark job requests most of the cluster memory, smaller jobs may remain waiting in the queue for a long time. This situation is known as **resource starvation**.

The problem may be caused by inappropriate scheduler configuration, missing queue limits, incorrect resource requests, or lack of fair resource allocation.

Likely Scheduling Errors

1. Incorrect scheduler configuration.
2. No separate queues for different users or departments.
3. One large application consumes excessive resources.
4. Missing maximum resource limits.
5. Incorrect CPU and memory allocation.
6. No user or application quotas.
7. Poor queue prioritization.
8. Lack of monitoring.
9. Resource requests are much larger than actual requirements.
10. Inappropriate scheduling policy for the workload.

Better Resource Management Approach

A suitable YARN scheduler should be configured according to the organization's workload.

Capacity Scheduler can be used when different departments or teams require guaranteed portions of cluster resources. For example:

- Data Analytics → 40%
- Research → 30%
- Development → 20%
- Testing → 10%

Resource limits can be applied to prevent one queue from consuming the entire cluster.

A **Fair Scheduler** approach can also be used when the objective is to provide fair resource sharing among users and applications.

Corrective Actions

- Configure Capacity Scheduler or an appropriate fair-sharing scheduler.
- Create separate queues for different users or teams.
- Assign minimum and maximum capacity to each queue.
- Set resource limits for individual applications.
- Configure user and group-level limits.
- Monitor CPU, memory, containers, and queue utilization.
- Optimize jobs to reduce unnecessary resource consumption.
- Prevent users from requesting significantly more resources than required.
- Prioritize critical workloads appropriately.
- Regularly review scheduler configuration.

Expected Result

With proper scheduling and resource management, no single user or job can monopolize the cluster. Resources are distributed fairly, waiting time is reduced, and multiple users can execute jobs concurrently. This improves overall cluster utilization and system responsiveness.

5. Backup Strategy Restores Outdated Data After Recovery

Problem Analysis

A distributed storage system should maintain reliable and up-to-date backups so that data can be restored after hardware failure, accidental deletion, corruption, or other disasters.

If recovery restores outdated data, the backup strategy may be taking backups too infrequently or using an old backup copy. Another possibility is that replication is being incorrectly considered as a backup. Replication protects against some hardware failures, but if incorrect or deleted data is replicated, the error may also be copied to all replicas.

An asynchronous replication system can also have **replication lag**, meaning the backup copy is behind the primary system.

Likely Design Flaws

1. Backups are performed too infrequently.
2. Only one old backup version is retained.

3. Replication is incorrectly treated as a complete backup.
4. Replication lag is not monitored.
5. Backup jobs are failing without being detected.
6. Backup integrity is not verified.
7. No incremental backups are maintained.
8. No point-in-time recovery mechanism exists.
9. Recovery procedures are not tested regularly.
10. Recovery Point Objective (RPO) is not properly defined.

Corrective Actions

- Perform regular full and incremental backups.
- Maintain multiple backup versions.
- Use snapshots or versioned backups where supported.
- Monitor backup completion and failures.
- Monitor replication lag.
- Keep backups in a separate failure domain or location.
- Maintain off-site copies for disaster recovery.
- Regularly verify backup integrity.
- Perform periodic restore tests.
- Define an appropriate **Recovery Point Objective (RPO)**.
- Define a **Recovery Time Objective (RTO)**.
- Keep historical versions so that recent valid data can be recovered.
- Do not depend only on synchronous or asynchronous replication as the backup mechanism.

RPO and RTO

RPO (Recovery Point Objective):
Defines how much recent data the organization can afford to lose. For example, an RPO of 15 minutes means backups or replication should allow recovery to a point no more than approximately 15 minutes behind the failure.

RTO (Recovery Time Objective):
Defines how quickly the system should be restored after a failure.

Example

Suppose the latest valid database update occurred at 10:00 AM, but the backup was taken at 6:00 AM. If the system fails at 10:30 AM and only the 6:00 AM backup is available, four and a half hours of changes may be lost.

A better strategy would use frequent incremental backups or snapshots so that recovery can use a recent recovery point.

Expected Result

A well-designed backup and recovery strategy ensures that the latest valid version of data can be restored after a failure. Multiple backup copies, versioning, regular verification, and clearly defined RPO/RTO values provide better data protection and disaster recovery capability.